

BOS-2L: Delay-Tolerant Overlap Scheduling with Guarded LP Rounding

A Two-Layer Recovery Architecture with a Conditional
 $2 - 1/m + \varepsilon$ Guarantee

Chen-Hong

Department of Computer Science and Information Engineering
National Central University, Taoyuan, Taiwan

September 27, 2026 — Research manuscript, version 20260927

Abstract

Bottleneck-Oriented Overlap Scheduling (BOS) prioritizes jobs using overlap-adjusted bottom levels on identical parallel machines. Its nominal nonpreemptive model does not describe a job that must pause when a required material fails to arrive. We extend BOS with a second recovery layer. The first layer executes a nominal BOS schedule. At a single disruption-revelation time, the second layer freezes completed processing, constructs a residual progress-checkpoint DAG, solves a time-indexed start-variable LP, and converts its fractional solution into a feasible chunk schedule by median-priority list decoding. A residual greedy candidate and a reference replay candidate protect against poor LP decoding. Let B be a workload/-path lower bound, D the maximum remaining material wait at revelation, and E the total duration inflation from grid rounding. We prove $C_{2L} \leq (2 - 1/m)B + D + E$. Consequently, if $D + E \leq \varepsilon B$, the architecture is a $(2 - 1/m + \varepsilon)$ approximation relative to the clairvoyant optimum of the disturbed progress model. The guarantee comes from reference replay and candidate selection, not an unproved integrality-gap claim. In 288 synthetic recovery cases, all feasibility and bound checks pass. LP decoding alone beats residual greedy in 7 cases and loses in 95; the portfolio retains the 7 improvements without the losses. All inputs, LP supports, schedules, exact-solver statuses, and code are supplied. These results establish a reproducible feasibility study, not general empirical superiority of LP rounding.

Keywords: overlap scheduling; material delays; preemption; residual DAG; LP rounding; predictive-reactive scheduling.

1 Motivation and scope

A short nominal makespan is not sufficient when production relies on materials that can arrive late. A machine may have completed useful work on a job before it reaches a missing-material gate. Restarting the entire job discards that progress; letting the successor start merely because wall-clock time passed can create nonexistent output. A recovery model must retain completed work, represent progress-based dependencies, and make explicit how much disruption a delivery-time guarantee tolerates.

The proposed architecture retains the nominal BOS mechanism and introduces a separate recovery problem only when a disruption is revealed. This follows the general predictive-reactive scheduling viewpoint [3]; it is not a claim that reactive scheduling or overlap dependencies are new. Our concrete contribution is a proof-carrying implementation that connects BOS, checkpoint recovery, a specified LP-to-schedule decoder, and an explicit disturbance budget.

What is established. This manuscript provides (i) a progress-consistent residual transformation; (ii) an executable LP rounding procedure; (iii) a reference-replay lemma and conditional global guarantee; and (iv) reproducible synthetic experiments including negative LP results. The theory is for identical machines, one revelation, and zero setup/migration cost. The reported experiments do not establish that LP is the best second-layer method.

Version lineage. The supplied September 15 BOS draft [1] is the nominal-model source. Its start–start lags, bottom-level recurrence and work-conserving proof are retained. The supplied research-plan slides motivate asynchronous arrivals and mid-job recovery, but their informal partial-kit and dynamic-input statements are not treated as existing theorems. The September 15 experimental counts are not reused as evidence for this new model. All experimental numbers below come from newly executed code.

2 Nominal BOS and its lower bound

Let $G = (V, A)$ be a DAG with processing times $p_i > 0$ and m identical unit-speed machines. All nominal releases are zero. A job runs nonpreemptively on one machine. For edge (i, j) , define

$$0 < \ell_{ij} \leq p_i, \quad \delta_{ij} = p_i - \ell_{ij}. \quad (1)$$

The strict positivity restriction is intentional in this version. The nominal constraints are $s_j \geq s_i + \ell_{ij}$ and machine nonoverlap. Zero-lag edges require an explicit first-start convention and are excluded from the recovery implementation; the September 15 model supported them.

For each path $P = (v_1, \dots, v_k)$, including singletons, let

$$Q(P) = \sum_{h < k} \ell_{v_h v_{h+1}} + p_{v_k}, \quad L = \max_P Q(P), \quad W = \sum_i p_i, \quad B = \max\{W/m, L\}. \quad (2)$$

BOS uses the static score

$$b_i = \max \left\{ p_i, \max_{(i,j) \in A} (\ell_{ij} + b_j) \right\} \quad (3)$$

and starts an executable maximum-score job whenever a machine is idle. Equal-time events are closed before time advances. The score is static for a fixed input; the executable set changes over time.

Lemma 1 (Nominal lower bound). $B \leq \text{OPT}_0$, where OPT_0 is the nominal nonpreemptive optimum.

Proof. The workload needs at least W/m time. Repeated substitution of the start constraints along a path yields $s_{v_k} + p_{v_k} \geq Q(P)$. Maximize over paths. $\square$

Theorem 2 (Nominal list bound). For the nominal BOS schedule S^0 with makespan C_0 ,

$$C_0 \leq W/m + (1 - 1/m)L \leq \rho B, \quad \rho = 2 - 1/m. \quad (4)$$

This holds for every work-conserving priority rule in this model.

Proof. Let $q(t)$ be the number of busy machines and $I = \int_0^{C_0} (m - q(t)) dt$. Choose a last-finishing job. For a selected job v , its eligibility time is $a(v) = \max(\{0\} \cup \{s_u + \ell_{uv} : (u, v) \in A\})$. If $a(v) > 0$, choose a predecessor attaining it and repeat backwards; otherwise terminate. This yields a single directed path $P = (v_1, \dots, v_k)$.

At a non-full time $t < s_v$, work conservation implies $t < a(v)$: otherwise v was executable while a machine was idle. Following the chosen predecessor either places t in its execution prefix $[s_u, s_u + \ell_{uv})$ or moves the same argument backwards. Thus all non-full times are covered by the

prefixes along P and the final interval $[s_{v_k}, s_{v_k} + p_{v_k})$. They are disjoint because every successor starts after its preceding prefix ends. Each has a running chain job since $\ell_{uv} \leq p_u$. Their total length is $Q(P) \leq L$, so $I \leq (m - 1)L$. Finally $mC_0 = W + I$. Applying $W/m \leq B$ and $L \leq B$ proves the result. No summation over different blocking paths is used. $\square$

This is a list-scheduling guarantee, consistent with the classical role of work conservation in Graham's analysis [2]. It does not certify a BOS-specific advantage over other priorities.

3 The disturbed progress model

3.1 Execution semantics

In the recovery model, a job may pause and migrate, but it never executes on two machines simultaneously. Let $u_i(t) \in \{0, 1\}$ indicate productive execution and define cumulative progress

$$x_i(t) = \int_0^t u_i(z) dz, \quad 0 \leq x_i(t) \leq p_i, \quad \sum_i u_i(t) \leq m. \quad (5)$$

An edge (i, j) permits the first execution of j only after $x_i(t) \geq \ell_{ij}$. Every incoming condition is required (AND semantics). This model does not mean that any arbitrary subset of materials is sufficient.

A material gate (i, g, a) means that job i cannot process beyond progress g , with $0 \leq g < p_i$, before absolute time a . Work strictly before g remains usable. At a gate one may pause i , run another available job, and later resume i . Material arrivals are permanent; they do not expire or get consumed by other jobs in this model.

Without interruptions, $x_i(t) = \min\{p_i, \max\{0, t - s_i\}\}$, so the positive-threshold progress condition is equivalent to the nominal start–start lag. With interruptions it is not: the clock can advance while progress stays fixed.

3.2 Revelation contract

At time $\tau < C_0$, the realized prefix is exactly the corresponding prefix of S^0 . All relevant late gates and their actual future arrival times are revealed at τ , or certified upper bounds are used conservatively instead. No gate has already been violated. Running jobs can be stopped at τ without lost work. After this revelation there are no further unknown delays, processing-time changes, breakdowns, or new jobs. Define

$$D = \max(\{0\} \cup \{a - \tau : a \text{ a revealed arrival } a\}). \quad (6)$$

Thus all revealed missing materials are available by $\tau + D$. D is a conservative remaining-wait envelope; it is neither the sum of delays nor the measured idle time of the selected schedule. The first layer need not know D in advance. An advance guarantee for a target ε requires an a priori bound on it.

3.3 Benchmark and tolerance

Let OPT_D be the clairvoyant optimum for the disturbed progress model from time zero, with all gates known, allowing arbitrary lossless preemption and migration. It need not share the BOS prefix or use the selected checkpoints. We also report a more restricted, frozen-prefix grid optimum in experiments; these benchmarks are distinct.

Lemma 3 (Lower bound survives recovery). $B \leq \text{OPT}_D$.

Proof. The total work is unchanged, giving W/m . A job cannot accumulate ℓ_{ij} processing in less than ℓ_{ij} elapsed time after its first execution because its rate is at most one. Hence first-start times still satisfy $s_j \geq s_i + \ell_{ij}$, and completion of the last path job takes at least p_{v_k} additional time. The same path bound L follows, regardless of preemptions or gates. $\square$

For a chosen dimensionless tolerance $\varepsilon > 0$, the time budget is εB . We will require that the material envelope and discretization inflation fit inside this budget. Choosing an arbitrarily small symbol ε does not make an arbitrary delay admissible.

4 Residual checkpoint DAG

For each job, collect 0, p_i , all outgoing thresholds ℓ_{ij} , all material-gate positions, and the executed progress $x_i(\tau)$. Consecutive cut positions define chunks. Remove completed chunks and retain only intervals after $x_i(\tau)$. A running job therefore contributes only its unprocessed suffix.

Each residual chunk a has original productive duration $d_a > 0$, job identity, progress interval, nominal absolute start t_a^0 , and nominal machine μ_a^0 . Consecutive chunks of one job are connected by finish–start edges. If a positive predecessor threshold is not yet attained and its successor job is unstarted, connect the chunk ending at that threshold to the first residual chunk of the successor. Already satisfied threshold edges are removed. Each material gate releases every downstream chunk of that job no earlier than its arrival; direct release bounds on all downstream chunks are redundant but convenient.

Let $H_\tau = (U, F)$ denote this DAG. Original cross-job edges respect the original acyclic order and internal edges increase progress, so no cycle is introduced. Migration is permitted between chunks; job-internal chain edges prevent parallel processing of one job.

Proposition 4 (Residual feasibility). *Every feasible finish–start schedule of H_τ appended to the frozen prefix is feasible for the disturbed progress model, provided chunk processing retains its progress order and gate releases are enforced.*

Proof. The cut at $x_i(\tau)$ preserves exactly the completed work. The internal chain gives each job sequential execution. Every unsatisfied positive threshold has a chunk ending at precisely that threshold, and its edge prevents the successor from starting early. Every satisfied threshold remains satisfied because progress is permanent. Releases prevent processing beyond an unavailable material gate. Chunk machine nonoverlap establishes global capacity. $\square$

The checkpoint model restricts when further preemption occurs; it does not assert that every arbitrary preemptive schedule has the same chunk representation. This matters when interpreting exact results.

5 LP formulation and explicit rounding

5.1 Time discretization

Choose grid width $h > 0$ and reserve

$$q_a = h \lceil d_a/h \rceil, \quad k_a = q_a/h, \quad R_a = \lceil \max\{0, r_a - \tau\}/h \rceil, \quad E = \sum_{a \in U} (q_a - d_a). \quad (7)$$

Here r_a is the absolute gate-release bound. In the implementation a chunk stores its release relative to τ . Padding $q_a - d_a$ reserves machine time but creates no productive progress. Threshold completion may be conservatively deferred to the end of its reservation. If $K = |U|$, then $0 \leq E < Kh$. When durations are grid-aligned, $E = 0$.

5.2 Start-variable relaxation

A safe integer horizon in grid units is $T = \max_a R_a + \sum_a k_a$, because all chunks can be run serially in topological order after all releases. For $t \in \{R_a, \dots, T - k_a\}$, y_{at} denotes the fractional start mass of chunk a at t . Let z represent makespan in grid units. Solve

$$\begin{aligned} \min \quad & z \\ \sum_t y_{at} &= 1 & a \in U, \end{aligned} \tag{8}$$

$$\sum_a \sum_{t: t \leq v < t+k_a} y_{at} \leq m \quad v = 0, \dots, T-1, \tag{9}$$

$$\sum_t t y_{at} + k_a \leq \sum_t t y_{bt} \quad (a, b) \in F, \tag{10}$$

$$\sum_t (t + k_a) y_{at} \leq z \quad a \in U, \tag{11}$$

$$0 \leq y_{at} \leq 1, \quad 0 \leq z \leq T.$$

Constraint (10) enforces precedence only in expectation. A fractional solution is not an executable preemptive schedule. In particular, fractional mass cannot be interpreted as physically completed material. This is a deliberately weak relaxation, useful for a reproducible initial decoder and a grid-model lower bound.

With binary y , the formulation is exact for the padded grid-chunk problem with a frozen prefix: each chunk selects one start, (10) becomes actual finish–start precedence, and capacity bounds the maximum number of overlapping intervals by m . Such intervals can be assigned to identical machines greedily. The fractional optimum $\tau + hz^*$ lower-bounds that restricted integer problem; padding and a frozen prefix mean it is not automatically a lower bound on unrestricted OPT_D .

5.3 Where rounding actually happens

For a fixed $\alpha = 1/2$, compute the start-mass median

$$\theta_a = \min \left\{ t : \sum_{s \leq t} y_{as}^* \geq \alpha \right\}. \tag{12}$$

Whenever a machine becomes idle, among released chunks with all predecessors completed, start the chunk minimizing (θ_a, a) . If none is ready, advance to the next completion or release event. This converts fractional start distributions into integral chunk starts and machine assignments. It is a deterministic priority decoder inspired by LP/quantile scheduling ideas [5]; it does not inherit the approximation factors from single-machine weighted-completion-time results.

The decoder does *not* round each y_{at} independently to zero or one, and θ_a is not a mandated start time. Readiness and capacity always override priority. Preemption allows a job to be split into chunks; rounding converts a fractional optimization solution into decisions. They are separate operations. Assignment rounding for independent jobs [4] also cannot be transferred to this DAG merely by citing its name.

Proposition 5 (LP decoder is feasible). *If the LP solver returns a feasible solution, the median decoder terminates and yields a feasible padded chunk schedule.*

Proof. Every dispatch requires a free machine, a satisfied release, and completion of all predecessor chunks. Each chunk is removed from the pending set exactly once. If chunks remain, either one is running or a source of the pending induced DAG has no pending predecessors and eventually reaches its finite release. Thus there is no deadlock. Residual feasibility then applies. $\square$

No bound of the form $C_{\text{LP-decoded}} \leq C_{\text{LP}} + \varepsilon B$ is proved or assumed. Expected-precedence relaxations can lose important temporal structure, and our experiments explicitly test that risk.

6 Guarded second-layer architecture

The second layer constructs three candidates using the same frozen prefix, checkpoint set, padded durations, and material gates:

1. **LP candidate P** : median-priority decoding of the LP solution.
2. **Greedy candidate G** : list scheduling by descending residual bottom level $\beta_a = q_a + \max_{(a,b) \in F} \beta_b$.
3. **Reference candidate F_0** : preserve nominal machine order and nominal offsets, then replay after the material envelope. This candidate provides the proof certificate.

Candidates P and G use grid-rounded releases and starts relative to τ . The reference candidate can start off-grid. Every candidate respects the same physical model; only the reference deliberately uses a more conservative timing rule.

6.1 Constructing the reference candidate

Add edges between successive residual chunks on each nominal machine, ordered by t_a^0 . These reference edges join nonoverlapping nominal intervals, so their union with F is acyclic. For each chunk in topological order set

$$f_a = \max \left\{ t_a^0 + D, \max_{b \rightarrow a} (f_b + q_b) \right\}. \quad (13)$$

Assign chunk a to μ_a^0 . All $t_a^0 \geq \tau$, so every chunk starts after $\tau + D$ and all material gates are satisfied. Original machine order and checkpoint precedences are respected.

Lemma 6 (Reference replay bound). *The reference schedule satisfies $C_{F_0} \leq C_0 + D + E$.*

Proof. First consider the same augmented DAG with durations d_a . The nominal residual times satisfy every arc and offset: for any arc $b \rightarrow a$, $t_a^0 \geq t_b^0 + d_b$. Hence the vector $t_a^0 + D$ is feasible and the earliest schedule has makespan at most $C_0 + D$.

For padded durations, express (13) as a longest-path recursion. Every path starts at one source offset $t_v^0 + D$ and adds durations of its vertices (including the final chunk when computing completion). Padding increases a given path's length by the sum of $q_a - d_a$ on that path. Acyclicity ensures each chunk appears at most once, so that sum is at most E . Taking the maximum over paths yields $C_{F_0} \leq C_0 + D + E$. This is a bound on a single maximizing path, not a sum of all idle intervals. $\square$

6.2 Selection and theorem

The architecture returns the feasible candidate with smallest makespan:

$$C_{2L} = \min\{C_{F_0}, C_G, C_P\}. \quad (14)$$

If the LP fails or times out, omit P . On ties prefer G to avoid attributing a gain to LP when it made none. Candidate computation is modeled as having zero schedule latency; deployment latency is discussed below.

Theorem 7 (Delay-tolerant BOS-2L). *Under the nominal, progress, and single-revelation assumptions stated above,*

$$C_{2L} \leq C_0 + D + E \leq \rho B + D + E. \quad (15)$$

For any prescribed $\varepsilon > 0$ satisfying

$$D + E \leq \varepsilon B, \quad (16)$$

we obtain

$$C_{2L} \leq (2 - 1/m + \varepsilon)B \leq (2 - 1/m + \varepsilon) \text{OPT}_D. \quad (17)$$

The same numerical guarantee relative to OPT_0 follows from $B \leq \text{OPT}_0$.

Proof. Selection makes $C_{2L} \leq C_{F_0}$. Apply the replay lemma, nominal bound, budget inequality, and disturbed-model lower bound, in that order. $\square$

Meaning of the theorem. The guaranteed factor is conditional on a bounded disturbance class, not on LP always rounding well. The portfolio cannot be worse than its feasible greedy candidate in makespan, and its replay certificate survives any LP priority quality. It can incur more computation. This construction is a safeguard-based approximation architecture, not a new unconstrained approximation ratio for general online scheduling.

Selecting a budget. Split $\varepsilon = \varepsilon_D + \varepsilon_Q$, require $D \leq \varepsilon_D B$, and choose $h \leq \varepsilon_Q B/K$ when $K > 0$. Then $E < Kh \leq \varepsilon_Q B$. The sharper admission check uses actual E . If all data are grid-aligned, $E = 0$ even for a nonzero grid width. If the budget fails, the feasible schedule and additive bound remain valid; the requested fixed- ε claim must be marked uncertified. Reporting $\hat{\varepsilon} = (D + E)/B$ after observing a case is a diagnostic, not evidence that a smaller preselected tolerance was met.

Computational cost. The nominal algorithm is event-driven and polynomial in the graph size. The residual LP has $O(KT)$ variables and $O(T + |F| + K)$ constraints in a sparse representation. Since T depends on processing-time magnitudes and $1/h$, this implementation is pseudo-polynomial and is not a strongly polynomial approximation scheme. Grid refinement trades time resolution for optimization cost.

7 A complete recovery example

Consider case 20 in the supplied trace set: $m = 2$, processing times $(5, 8, 8, 5, 6, 8)$, and edges $(0, 4, 4)$, $(1, 5, 6)$, $(2, 5, 6)$, $(4, 5, 2)$, where each triple is (predecessor, successor, threshold). The nominal BOS starts are $(0, 0, 5, 14, 8, 13)$ and $C_0 = 21$. Total work is 40, and $B = 20$.

At $\tau = 6$, job 1 has completed six units and its remaining two units require a material arriving at time 9. Job 2 has completed one unit. Both running jobs are split at their current progress; their completed portions remain fixed. With $h = 1$, $D = 3$ and $E = 0$. Table 1 gives the actual LP-decoded recovery. Job labels are zero-based, matching the code.

Table 1: Case 20: LP recovery after the unchanged prefix $[0, 6)$.

Machine	Start	End	Job / progress interval
0	6	11	Job 2: $[1, 6]$
0	11	13	Job 2: $[6, 8]$
0	13	15	Job 1: $[6, 8]$
0	15	20	Job 3: $[0, 5]$
1	6	8	Job 4: $[0, 2]$
1	8	12	Job 4: $[2, 6]$
1	12	20	Job 5: $[0, 8]$

The delayed machine starts job 4 instead of waiting for job 1’s material. Job 1 resumes later with its original six units intact. Job 5 starts only after all three progress thresholds are satisfied. The LP has five fractional chunk distributions; its objective corresponds to absolute time 19, while the decoded schedule finishes at 20. Thus the LP solution is not itself an executable schedule. The workload bound proves that this particular completed schedule is globally optimal for the disturbed instance. Greedy repair finishes at 21 and replay at 24. The certified tolerance is $\hat{\varepsilon} = 3/20 = 0.15$, giving the conservative bound $(1.5 + 0.15)20 = 33$. Recovery can improve over nominal C_0 because the feasible policy now permits preemption; this must not be interpreted as a delay improving the physical instance.

8 Reproducible experiments

8.1 Design and audit trail

We generated $2 \times 2 \times 2 \times 6 \times 3 \times 2 = 288$ recovery cases: two graph families, $n \in \{6, 10\}$, $m \in \{2, 3\}$, six seeds, delay lengths $\{1, 3, 6\}$, and grid widths $h \in \{1, 2\}$. Processing times are uniformly sampled integers from 4 through 10. In the layered family each ordered pair $i < j$ receives an edge independently with probability 0.25. The merge family uses probability 0.6 for edges into the last two jobs and 0.07 otherwise. Each included edge has lag $\max\{1, \text{round}(p_i a)\}$ for $a \in \{0.25, 0.5, 0.75\}$ sampled uniformly.

The trigger is $\tau = \max\{1, \lfloor 0.3C_0 \rfloor\}$. We select the first indexed job strictly in progress and place a gate exactly at its current progress. If none is in progress, the trigger is one time unit after the first nominal job starts. Its gate arrival is $\tau + \text{delay}$. This ensures every experiment includes an actually interrupted running job and a feasible unchanged prefix. Each case has one late gate; multi-gate experiments and real production traces remain future work.

All methods use the same prefix and residual instance. We compare reference replay, residual greedy, LP decoding, guarded LP $\min\{P, F_0\}$, and the full portfolio. All generated instances, LP positive supports, selected medians, chunk schedules, and nominal schedules are recorded. The validation routine checks releases, precedences, machine nonoverlap, makespan, and both replay and global certificate inequalities. Any failure aborts the experiment rather than being excluded. A separate audit reconstructs the complete schedules from the original jobs, including the frozen prefix, and checks productive progress, material gates, and machine reservations independently of the chunk edges. All 1,152 audited candidate/selected schedules pass; selected schedules can duplicate candidates.

8.2 Main results

All 288 LPs solved successfully. In 284 cases at least one chunk had more than one positive start variable, confirming that the experiment actually exercised fractional-to-integral decoding. There were zero feasibility or certificate violations across the compared schedules.

Table 2: Paired makespan comparison against residual greedy (288 cases).

Method	Win	Tie	Loss
LP decoding alone	7	186	95
Full BOS-2L portfolio	7	281	0

The portfolio’s mean paired improvement over greedy is 0.141%, defined as the average of $100(C_G - C_{2L})/C_G$. Its mean improvement over conservative replay is 14.037%. LP alone has negative mean paired improvement of approximately -1.95% against greedy. The full portfolio selects LP only in the seven strictly improving cases. These results argue against LP-only recovery as the default choice for these instances.

There are 144 cases per grid width; LP adds four improvements for $h = 1$ and three for $h = 2$. The observed budget requirement $\hat{\varepsilon}$ ranges from 0.0238 to 0.9273. For an illustrative fixed target $\varepsilon = 0.2$, only 124 of 288 cases meet the sufficient budget condition (101 at $h = 1$, 23 at $h = 2$). The remaining cases retain an additive certificate, not a certified $\rho + 0.2$ ratio. This distinction prevents a post hoc fitted tolerance from being reported as an advance guarantee.

8.3 Exact subset

For $n = 6$, the first two seeds, and $h = 1$, all 24 tested frozen-prefix integer models reached solver status optimal and zero MIP gap. This subset was specified by index, not selected for favorable outcomes. Table 3 reports mean total-makespan gaps, $100(C - C_{\text{grid,prefix}}^*)/C_{\text{grid,prefix}}^*$.

Table 3: Mean gaps to 24 certified frozen-prefix grid optima.

Method	Mean gap (%)
Reference replay	20.088
Residual greedy	0.556
LP decoding	2.394
Guarded LP	2.394
Full BOS-2L portfolio	0.278

These are exact optima for the declared restricted recovery model, not proofs of unrestricted disturbed-instance optimality. Here $h = 1$ gives zero duration padding, but the frozen prefix and checkpoint restrictions still matter.

8.4 Runtime and limits of the evidence

The run used Python 3.12.14, NumPy 2.3.5, and SciPy 1.17.0 with HiGHS. Mean LP solve time was about 4.2 ms on the execution environment, excluding model construction and candidate validation. The whole 288-case experiment, including 24 exact solves and trace output, took about 2.53 s in this run. These are small-instance measurements and not deployment latency guarantees. Repeated scenarios share nominal graphs and are correlated; no independent-sample significance claim is made.

The portfolio was adopted after an exploratory run showed LP-only weaknesses on this same suite. Consequently the final suite is a development evaluation, not an untouched held-out test. Its zero-loss property versus greedy follows from selection by construction; it is not an empirical discovery. A larger held-out suite, alternative LP relaxations, equal-time-budget heuristics, and multi-delay data are needed before making a strong practical recommendation.

9 Discussion, limitations, and next steps

Why LP remains in the architecture. It provides a concrete alternative ordering and a restricted-model lower bound, and it improves seven generated cases. However, the current expected-precedence LP is weak and the median decoder loses useful information. Strengthening cumulative precedence constraints, trying several deterministic quantiles, and using local search are plausible next experiments; their performance is not established here. A cheap greedy-only second layer remains a serious baseline.

Not an automatic buffer optimizer. The user-facing motivation is delivery flexibility. This version certifies an admissible delay envelope while still optimizing makespan among candidates. It does not maximize the tolerable delay region or optimize an overlap-utilization objective. Calling it an optimal robustness-allocation algorithm would overstate the result. A next formulation could fix a delivery deadline and maximize admissible gate delays, but would require separate definitions and proofs.

Costs and repeated uncertainty. Arbitrary interruptions, setup costs, migration overhead, machine eligibility, speed heterogeneity, and later unannounced arrivals invalidate the present assumptions. A bounded global computation pause T_{solve} can be charged additively by delaying the already certified recovery, yielding $C \leq \rho B + D + E + T_{\text{solve}}$ if no additional disruptions occur. More complex costs need an explicit feasible reference construction. Repeated application after committing an LP suffix does not automatically inherit the original reference proof: the new prefix is no longer necessarily a prefix of S^0 .

Necessity of bounded disturbance for a nominal-relative claim. One unit job with a gate at zero and arrival R has nominal optimum 1 and disturbed completion at least $R + 1$. No fixed nominal-relative factor independent of R can hold. This counterexample concerns the nominal benchmark; it does not prove impossibility relative to the disturbed optimum, which also grows with R .

Novelty boundary. Temporal precedence, predictive-reactive scheduling, LP-guided priorities, and conservative fallback selection each have precedents. The defensible contribution at this stage is their explicit BOS-specific progress transformation, certificate, and reproducible examination. A claim of first use or state-of-the-art performance would require a broader literature review and stronger evidence.

10 Conclusion

BOS-2L separates nominal overlap scheduling from delayed-material recovery. The second layer preserves progress, solves and decodes an explicit LP, and compares the result against greedy repair and certified replay. The bound $\rho B + D + E$ establishes a conditional $2 - 1/m + \varepsilon$ guarantee when the disturbance and grid inflation fit the declared tolerance budget. The experiments show why the safeguard and greedy candidate are essential: the present LP decoder is feasible but often worse than greedy. The manuscript therefore supports a bounded-delay recovery architecture with inspectable code, rather than an unconditional LP-rounding approximation claim.

A Algorithm and reproduction

1. Run nominal BOS; compute B , save S^0 , and execute its prefix.
2. At first revelation τ , check prefix feasibility and record all arrival times.
3. Split at progress thresholds, material gates and current progress; remove completed work.
4. Choose h , compute q_a , E , D , and budget status $D + E \leq \varepsilon B$.
5. Construct replay by (13); construct residual greedy.
6. Solve (8)–(11), compute medians, and list-decode if successful.
7. Validate each candidate and choose the smallest makespan. Keep the additive certificate even if the target budget is exceeded.

The public entry points are `residualize`, `lp_model`, `lp_round`, `decode`, `fallback`, `validate`, and `solve` in `code/two_layer.py`. The exact binary formulation is implemented in `exact`. The retained `bos_core_0915.py` is the supplied nominal engine, copied without mathematical changes. Run:

```
cd code
python run_experiments.py --seeds 6 --out ../results_rerun
```

`results/raw.csv` contains all 288 rows and `results/traces/` contains one JSON record per case. `summary.json` records package versions, counts, gaps, and timings. The proof uses exact arithmetic; implementation assertions use tolerance 10^{-7} and solver floating-point output. Runtime and fractional supports may differ across solver versions even when makespans agree.

B Checks that prevent misleading conclusions

- A fractional LP allocation is never dispatched as physical production.
- The completed prefix is never rerun or silently optimized away.
- Original jobs cannot execute in parallel with themselves.
- The ratio uses original workload/path B ; the residual LP objective is not substituted for it.
- ε is dimensionless; D , E , and εB are time quantities.

- Solver timeouts are not labeled optimal; exact claims require status 0.
- The proof does not assume that the LP candidate beats the reference or greedy.
- The entire new data set is reported, including LP losses and failed budget admissions.

References

- [1] Chen-Hong, “BOS: Bottleneck-Oriented Overlap Scheduling on Identical Parallel Machines,” supplied research manuscript, September 15, 2026. Local sources: `BOS_greedy_20260915.pdf`, reproducible project, and `REVIEW_0914.md`.
- [2] R. L. Graham, “Bounds on Multiprocessing Timing Anomalies,” *SIAM Journal on Applied Mathematics*, vol. 17, no. 2, pp. 416–429, 1969. <https://doi.org/10.1137/0117039>.
- [3] W. Herroelen and R. Leus, “Project scheduling under uncertainty: Survey and research potentials,” *European Journal of Operational Research*, vol. 165, no. 2, pp. 289–306, 2005. <https://doi.org/10.1016/j.ejor.2004.04.002>.
- [4] D. B. Shmoys and E. Tardos, “An approximation algorithm for the generalized assignment problem,” *Mathematical Programming*, vol. 62, pp. 461–474, 1993. <https://www.cs.cornell.edu/~eva/Approx.Algorithm.Generalized.Assignment.Prob.pdf>.
- [5] M. Skutella, “List Scheduling in Order of α -Points on a Single Machine,” *Efficient Approximation and Online Algorithms*, Lecture Notes in Computer Science 3484, Springer, pp. 250–291, 2006. Author-hosted full text: <https://page.math.tu-berlin.de/~skutella/alpha.pdf>. Used for LP-priority background, not as a theorem for the present makespan model.